MPOO 2027-1 · GRUPO 06

# Parcial 1 · correcciones

Versión C del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

**Antes de empezar.** Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_castillo_angel
git pull --no-rebase
```

**Qué tienes que hacer.** En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/castillo_angel/examen/p1/`, con tu push de siempre.

## Pregunta 0.1 · Preguntas abiertas

**Pregunta**

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

**Tu respuesta**

```
Es considerar unicamente las caracteristicas que nos son utiles del objeto, por ejemplo: si
vamos a hacer una television en la que solo esten las funciones de cambiar de canal nos
convendria considerar "el numero de canal" y despreciar la marca de la tele, ya que, no es un
atributo que nos sirva para ese caso.
```

**Corrección.** Correcta: define la abstracción como quedarse con lo útil para el problema y da un ejemplo justificado: conserva el número de canal y omite la marca porque no sirve en ese contexto.

## Pregunta 0.2 · Preguntas abiertas

**Pregunta**

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

**Tu respuesta**

```
Un constructor sirve para identificar que el atributo es de ESTE obejeto en especifico, se
distingue porque no requiere de un tipo de dato y es declara con this.
```

**Corrección.** Acierta en parte al decir que el constructor no lleva tipo (no declara retorno), pero confunde su propósito con el de this. El constructor inicializa el estado de la instancia al crearla, lleva el mismo nombre que la clase y no declara retorno, ni siquiera void.

## Pregunta 0.3 · Preguntas abiertas

**Pregunta**

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

**Tu respuesta**

*Sin respuesta.*

> **Corrección.** Sin respuesta.

## Pregunta 0.4 · Preguntas abiertas

**Pregunta**

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

**Tu respuesta**

```
== no basta porque con eso podemos comparar las direcciones o datos primitivos, para comparar el contenido de dos string nos convendria usar .equals( ).
```

> **Corrección.** Correcta: == compara direcciones (identidad) y para comparar el texto de dos String se usa equals.

## Pregunta 0.5 · Preguntas abiertas

**Pregunta**

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

**Tu respuesta**

```
En el do while: Sin importar la condicion entrara una vez a nuestro bucle, entro una vez y luego evalua la condicion.
Por otro lado: En un while desde el inicio se evalua la condicion, si esta es falsa, no entra al while.
```

> **Corrección.** Correcta: while evalúa la condición antes y puede no entrar nunca; do-while ejecuta el cuerpo una vez y después evalúa, aunque la condición sea falsa desde el inicio.

## R1 · ¿Qué imprime?

**Pregunta**

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{9, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla[0][i];
    System.out.println(i + ":" + total);
}
```

```
System.out.println((double) (total / 2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

**Tu respuesta**

```
0:9
1:13
2:20
10.00
P42
```

**Corrección.** La última línea P42 es correcta, y el formato i:total y la acumulación se reconocen en las líneas 1 a 3. Falló el alias: fila es la misma referencia que tabla[0], así que fila[i] / 2 cambia tabla[0] y la salida es 0:4, 1:6, 2:9. La línea 4 no es 10.00: Java imprime 10.0, y la salida correcta era 4.0 (9 / 2 = 4 y luego el cast).

**Qué escribir a mano.** La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

## R2 a) · ¿Compila, truena o funciona?

**Pregunta**

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
```

**Tu respuesta**

**Clasificación**

```
Compila pero truena
```

**Causa o salida**

```
El indice se sale en el for
```

**Corrección.** Correcto: compila pero truena porque i <= datos.length intenta leer datos[2], fuera del arreglo (ArrayIndexOutOfBoundsException).

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R2 b) · ¿Compila, truena o funciona?

**Pregunta**

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer piezas = 6;
System.out.println(piezas + 2);
```

```
System.out.println("P" + piezas);
```

**Tu respuesta**

**Clasificación**

```
Funciona
```

**Causa o salida**

```
8
P6
```

**Corrección.** La clasificación y la salida 8 / P6 son correctas. Faltó explicar la regla: en piezas + 2 el Integer se desempaqueta para sumar, y con un texto se concatena.

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R2 c) · ¿Compila, truena o funciona?

**Pregunta**

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
static int medir(int valor) {
    if (valor > 0) return valor;
}
```

**Tu respuesta**

**Clasificación**

```
No compila
```

**Causa o salida**

```
falta de {} en el if
```

**Corrección.** La clasificación 'no compila' es correcta, pero la causa no: las llaves del if son opcionales. No compila porque falta un return de int cuando valor es menor o igual que 0, y Java revisa todos los caminos.

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R3 · Modelado: base, reto y justificación

**Pregunta**

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

```
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
```

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

**Tu respuesta**

**Clase**

*Sin respuesta.*

**Justificaciones**

*Sin respuesta.*

> **Corrección.** Sin respuesta.

## R4 · Código con repetición

**Pregunta**

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

```
static int posicionMinima(int[] temperaturas)
```

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {7} → 0 · {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

**Tu respuesta**

```
int mtemp;
int pmtemp;
mtemp = temperaturas [0];
for( i = 0; i < temperaturas.length; i++){
    if(mtemp >= temperaturas [i]){
    mtemp = temperaturas[i];
    } else {
    pmtemp = i-1;
}
```

> **Corrección.** Recorre el arreglo con for y compara el mínimo guardado con cada elemento. Falta la cabecera static int posicionMinima(int[] temperaturas) y el return. La posición se guarda mal: pmtemp = i-1 queda en el else en lugar de pmtemp = i junto a la actualización del mínimo. Con >= devolvería la última repetición y no la primera, y con {7} pmtemp no se inicializa en 0; la sintaxis está incompleta por llaves sin cerrar e i sin declarar.

**Qué escribir a mano.** El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

## R5 · Código con selección

**Pregunta**

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

```
static String mantenimiento(int ciclos)
```

| Regla o valor recibido | Texto devuelto |
|---|---|
| Menor o igual que 0 | "INVALIDO" |
| Divisible entre 24 | "COMPLETO" |
| Si no, divisible entre 12 | "REVISION" |
| Si no, divisible entre 4 | "BASICO" |
| Cualquier otro | "NINGUNO" |

**Tu respuesta**

```
int ciclosing = ciclos;

if(ciclosing <= 0){
    System.out.println("INVALIDO");
} else if ((cilosing%24) == 0){
    System.out.println("COMPLETO");
} else if ((cilosing%12) == 0){
    System.out.println("REVISION");
} else if ((cilosing%4) == 0){
    System.out.println("BASICO");
} else {
    System.out.println("NINGUNO");
}
```

**Corrección.** La lógica y el orden son correctos: primero <= 0, luego 24, 12 y 4, y else NINGUNO. Falta la cabecera static String mantenimiento(int ciclos), imprime con println en vez de hacer return de cada texto aunque el enunciado prohíbe imprimir, y usa cilosing en vez de ciclosing.

**Qué escribir a mano.** El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.